



Unit 1 — Karel

Unit Test Study Guide · What You Need to Know

Name _____ Period _____ **Covers:** Unit 1 lessons 1.2–1.16 **Test:** 30 MC · 45 min

STUDY GUIDE

Everything on the Unit 1 Test

This guide mirrors the test exactly — every concept below maps to one or more of the 30 questions, and nothing on the test falls outside this guide. If you can do every bullet below without looking it up, you are ready. The test is 30 multiple-choice questions in 45 minutes: four choices each, one best answer, plus one *Select two* question. About a third of the questions ask you to read a short piece of code.

Karel quick reference (all the code on the test)

These are the only commands and conditions that appear on the test.

Type	Code	What it does
Command	<code>move()</code>	Karel moves forward one corner (crashes if a wall is in front).
Command	<code>turn_left()</code>	Karel turns 90° counterclockwise (to its left).
Command	<code>put_ball()</code>	Karel puts one ball on its current corner.
Command	<code>take_ball()</code>	Karel picks up one ball from its current corner.
Not built in	<code>turn_right()</code>	Basic Karel does <i>not</i> have this. Same effect = <code>turn_left()</code> three times. Super Karel has it built in.
Super Karel	<code>turn_right()</code> , <code>turn_around()</code>	Extra turning commands Super Karel already knows.
Condition	<code>front_is_clear()</code>	True/False — is there no wall in front of Karel? (A condition, not a command.)
Condition	<code>ball_present()</code> / <code>no_ball_present()</code>	True/False — is there / is there not a ball on Karel's corner?

The three building blocks (know these words)

- **Sequencing** — running statements one after another, top to bottom.
- **Selection** — choosing whether to run code, using `if` / `if...else`.
- **Iteration** — repeating code, using a `for` loop or a `while` loop.

1.2–1.3 — Karel’s world and the basic commands

- Describe Karel’s world: a **grid of streets (rows) and avenues (columns)** that Karel moves through.
- Name the **four commands** basic Karel knows with no functions defined: `move()`, `turn_left()`, `put_ball()`, `take_ball()`.
- Explain that a program runs **top to bottom in order** — this is **sequencing**.
- State what happens when Karel tries to `move()` into a wall: the program **stops with an error** (Karel does not turn, skip, or jump).
- Trace a short sequence of `move()` / `put_ball()` commands and count how many corners Karel travels and how many balls end up where. Example: `move()`, `put_ball()`, `move()` from the far-west corner facing East → Karel ends **2 corners east** with **1 ball** on the middle corner.

1.4 — turn_left and “Karel can’t turn right”

- State that `turn_left()` is exactly **90° counterclockwise** (not clockwise, not 180°, not 45°).
- Show that a right turn is the same as `turn_left()` **three times**.
- Give the reasons to **define a `turn_right()` function** instead of retyping three left turns everywhere: the code is **shorter, easier to read, and you can fix it in one place**. (It is not about speed, and the three-turn version does work.)

1.5 — Functions: defining vs. calling

- Explain the difference: **defining** a function describes what it does; **calling** it tells the program to run it now.
- Predict what happens to a function that is **defined but never called**: its code **does not run at all** (no crash, no automatic run).
- Count actions through a helper call. Example: if `turn_right()` is defined as three `turn_left()`s and `main()` calls `turn_right()` once, Karel turns left **3** times.
- Pick a **good function name**: one that describes the action, like `put_three_balls()` — not `f1()`, `do_stuff()`, or an existing command name.

1.6 — Top-down design and decomposition

- Define **top-down design**: break the big task into smaller subtasks, then write a function for each.
- Recognize **good decomposition**: `main()` is a short, readable **outline of calls to well-named helper functions** — not one long list of `move()` / `put_ball()`, and not one giant undefined function.

1.7 — Comments

- State what `#` does in Python-style Karel: it **begins a comment**, which the computer **ignores** when running the program.

1.8 — Abstraction

- Define **abstraction**: **hiding complex details behind a simple name** so you can use something without knowing exactly how it works.
- Explain why you can call a tested function like `build_tower()` **without reading what is inside it** — that is abstraction.
- Identify **everyday examples** of abstraction: driving a car with the wheel and pedals without knowing how the engine works; pressing an elevator button without knowing how the motor works.

1.9 — Super Karel

- State what **Super Karel** adds: built-in `turn_right()` and `turn_around()`. (It does *not* let Karel go through walls or skip `main()`.)

1.10 — For loops (iteration you count)

- Say how many times `for i in range(n):` runs its body: exactly **n** times. `range(5)` → 5 times (i = 0, 1, 2, 3, 4).
- Choose a `for` loop when you **know exactly how many times** to repeat.
- Recognize that `for i in range(10): move()` is the **iteration** way to move 10 times without retyping `move()`.

1.11–1.12, 1.14 — If, if/else, and conditions

- State what an `if` statement does: run a block of code **only when its condition is true**.
- Trace an `if/else`. Example: on a corner with **no ball**,

```
if ball_present():
    take_ball()
else:
    put_ball()
```

runs the `else` branch, so the corner ends with **1 ball**.

- Identify a valid **condition**: something that is true/false, like `front_is_clear()`. Commands (`move()`, `turn_left()`) and `range(5)` are **not** conditions.
- Match “do this **only if**...” wording to a **conditional / selection** (e.g. `if no_ball_present(): put_ball()`).

1.13 — While loops

- Say what `while condition:` does: repeat the body **as long as the condition stays true**.
- Choose a `while` loop when you **do not know how many times** to repeat — for example, move until you reach a wall: `while front_is_clear(): move()`.
- Explain the **infinite loop** danger: `while front_is_clear(): put_ball()` with **no** `move()` inside never changes the condition, so it runs forever.

1.15 — Debugging and error types

- Tell the **three kinds of errors** apart:
 - **Syntax error** — the code is written wrong and will not run.
 - **Runtime error / crash** — the program stops while running (e.g. Karel hits a wall).
 - **Logic error** — the program runs with no error message but produces the **wrong result**.
- Classify “runs fine but Karel stops one corner short every time” as a **logic error**.

1.16 — Algorithms

- Define an **algorithm**: a **finite, step-by-step set of instructions** for completing a task.
- State that **one task can be solved by different algorithms**, and that algorithms can be written in plain language, not just Python, and can use loops and conditionals.

Vocabulary checklist

Be able to define each term in one sentence: **sequencing, selection, iteration, function (procedure), function definition, function call, parameter/argument, top-down design, decomposition, abstraction, comment, condition, for loop, while loop, infinite loop, algorithm, syntax error, runtime error (crash), logic error, Super Karel, street, avenue.**

FASTEST WAY TO STUDY

Cover the right-hand column of the quick-reference table and name each command from memory. Then hand-trace these four: (1) `range(6)` — how many runs? (2) one call to a `turn_right()` helper — how many `turn_left()`s? (3) the `if/else` ball trace above. (4) `while front_is_clear(): put_ball()` — what goes wrong? If those feel easy, review the vocabulary checklist once and you’re done.